



Verteilte Systeme · Übungsblatt 2 · Sommer 2026

Thema: Schnappschuss eines konsistenten globalen Zustands

Im zweiten Teil der Portfolioprüfung geht es um die praktische Umsetzung eines **Schnappschuss-Algorithmus** (vgl. Vorlesung, Kapitel 4 *Algorithmen*, Abschnitt „Schnappschüsse“). Sie implementieren dazu eine kleine, klar umrissene verteilte Anwendung im Simulator **sim4da** und erfassen während ihrer Ausführung einen *konsistenten* globalen Zustand — bestehend aus den **lokalen Prozesszuständen** und den **unterwegs befindlichen Nachrichten**.

Anders als im ersten Übungsblatt soll *nicht* die verteilte UDP-Anwendung erweitert werden. Stattdessen starten Sie mit einer bewusst minimalen Anwendung, damit der Schnappschuss-Algorithmus selbst im Mittelpunkt steht.

Verwenden Sie den in der Vorlesung vorgestellten Simulator <https://github.com/syssoft-ds/sim4da-S26.git>. Als Ausgangspunkt eignen sich das Beispielprojekt `sim4da-S26-pingpong`, der Test `OneRingToRuleThemAll` sowie der Abschnitt `FIRST_SIMULATION.md`. Nachrichten sind als `record` umzusetzen (Marker-Interface `Message`); im `engage()` einer `Node` stehen `send`, `broadcast`, `receive` und `sleep` zur Verfügung.

Wichtige Hinweise zur Simulation

Damit überhaupt interessante, *inkonsistenzträchtige* Abläufe entstehen, müssen Sie das Standardverhalten des Simulators bewusst „verschärfen“:

- **Nachrichtenlatenz einführen.** Ohne Übertragungsverzögerung werden Nachrichten praktisch sofort zugestellt; das Zeitfenster, in dem eine Überweisung „unterwegs“ ist, ist dann verschwindend klein und die Kanalzustände bleiben fast immer leer. Modellieren Sie daher eine (zufällige) Latenz — etwa durch eine kurze, zufällig bemessene Verzögerung, die die Übertragungszeit nachbildet — so dass Nachrichten merklich lange im Transit sind. Andernfalls treten Inkonsistenzen nur sehr selten auf.

- **FIFO-Zustellung abschalten.** Per Voreinstellung liefert sim4da Nachrichten in Ankunftsreihenfolge (Kopf der Warteschlange) aus. Deaktivieren Sie diese FIFO-Eigenschaft, indem Sie eine Verteilungsfunktion für die Nachrichtenauswahl definieren, z. B.

```
SimulationBehavior.setMessageQueueSelectionDistributionFunction(  
    RandomValues.getUniformDistribution());
```

Damit wählt ein Empfänger eine *zufällige* Nachricht aus seiner Warteschlange — Nachrichten können also umsortiert zugestellt werden.

Aufgabe 1 – Eine kleine verteilte Anwendung (das „Bankhaus“)

Implementieren Sie in sim4da ein verteiltes Kontensystem aus n Prozessen, die vollständig vernetzt sind (jeder Prozess kann jedem anderen Nachrichten schicken).

- Jeder Prozess P_i verwaltet ein **Konto** mit einem ganzzahligen Saldo $balance_i$. Zu Beginn besitzt jeder Prozess denselben Startbetrag (z. B. 1000). Die **Gesamtsumme** $S = \sum balance_i$ ist damit zu Beginn bekannt.
- Wiederholt (z. B. nach einer kurzen, zufälligen Wartezeit) überweist ein Prozess einen **zufälligen Betrag** b (mit $0 < b \leq balance_i$) an einen **zufällig gewählten** anderen Prozess: Er reduziert sofort sein eigenes Konto um b und sendet eine Basisnachricht `Transfer(b)` an den Empfänger.
- Beim Empfang einer `Transfer(b)`-Nachricht erhöht der Empfänger sein Konto um b .

Die **Invariante** der Anwendung lautet: Die Summe aus allen Kontoständen *und* allen gerade unterwegs befindlichen Überweisungen ist konstant gleich S . Geld geht weder verloren noch entsteht neues — nur die Verteilung ändert sich. Genau diese Invariante werden Sie mit Ihrem Schnappschuss überprüfen.

Sorgen Sie dafür, dass die Anwendung „lange genug“ läuft (z. B. feste Anzahl Überweisungen pro Prozess oder externe Laufzeitbegrenzung über `simulator.simulate(seconds)`), damit ein Schnappschuss *während* des Betriebs ausgelöst werden kann.

Aufgabe 2 – Implementierung des Schnappschuss-Algorithmus

Erfassen Sie zur Laufzeit einen **konsistenten globalen Zustand** Ihres Bankhauses. Der aufgezeichnete Zustand muss umfassen:

1. den **lokalen Zustand** jedes Prozesses (seinen Kontostand zum Schnappschuss-Zeitpunkt) und
2. den **Kanalzustand** jeder Verbindung, also alle Überweisungen, die zum Zeitpunkt des Schnitts bereits abgeschickt, aber noch nicht empfangen wurden.

Sie dürfen **eine** der beiden folgenden Varianten umsetzen:

- **(a) Koordinator-/Einfärbeverfahren** wie in der Vorlesung: Ein Koordinator startet den Schnappschuss, indem er eine Kontrollnachricht (`state?`, Farbe) per Multicast an die Gruppe schickt. Empfänger wechseln die Farbe (weiß → schwarz), notieren ihren lokalen Zustand und melden ihn an den Koordinator zurück. Behandeln Sie dabei explizit die in der Vorlesung diskutierten Fälle der **Nachrichten-Einfärbung**: nachträgliche schwarze (Basis-)Nachrichten gehören noch zum gemeldeten Zustand und müssen erfasst werden; für den Fall „schwarzer Prozess empfängt weiße Nachricht“ wählen und begründen Sie eine der beiden Lösungsvarianten.
- **(b) Chandy-Lamport** (dezentral): Ein beliebiger Prozess initiiert den Schnappschuss, indem er seinen lokalen Zustand sichert und über **jeden** ausgehenden Kanal eine Marker-Nachricht sendet. Beim erstmaligen Empfang eines Markers sichert ein Prozess seinen lokalen Zustand und beginnt, die übrigen Eingangskanäle aufzuzeichnen; ein erneuter Marker auf einem Kanal beendet dessen Aufzeichnung. Sammeln Sie die Teilzustände abschließend ein.

Halten Sie in Ihrer Lösung die **Annahmen** des Verfahrens fest (vgl. Vorlesung: keine Nachrichtenverluste, keine Abstürze). Beachten Sie insbesondere, dass Sie die **FIFO-Zustellung abgeschaltet** haben (siehe Hinweise zur Simulation): Das klassische Chandy-Lamport-Verfahren *setzt FIFO-Kanäle voraus*. Diskutieren Sie, welche Konsequenzen das nicht-FIFO-Verhalten für Ihre gewählte Variante hat und wie Sie die Kanalaufzeichnung trotzdem korrekt umsetzen.

Am Ende soll der initiiierende Prozess bzw. der Koordinator den vollständigen globalen Zustand (alle Kontostände **und** alle Kanalinhalte) ausgeben.

Aufgabe 3 – Konsistenz nachweisen

Zeigen Sie experimentell, dass Ihr Schnappschuss tatsächlich einen **konsistenten** Schnitt liefert:

1. Lösen Sie Schnappschüsse **während des laufenden Betriebs** aus (das Überweisen geht weiter, während der Algorithmus läuft). Weisen Sie nach, dass die im Schnappschuss erfasste Gesamtsumme (Konten + Kanäle) **stets gleich S** ist.
2. Implementieren Sie zum Vergleich einen **naïven** Schnappschuss: Ein Koordinator fragt nur die Kontostände ab und ignoriert die Kanäle. Zeigen Sie, dass dieser einen **inkonsistenten** globalen Zustand liefern kann (Geld scheint zu „verschwinden“ oder zu „entstehen“, weil Überweisungen unterwegs nicht erfasst werden — eine „Nachricht aus der Zukunft“). Erklären Sie den Zusammenhang mit dem Begriff des konsistenten Schnitts ($e_2 \in S \wedge e_1 <_k e_2 \Rightarrow e_1 \in S$).
3. Variieren Sie `n` sowie die Überweisungsfrequenz und erfassen Sie einige statistische Größen (z. B. Anzahl der Kontrollnachrichten pro Schnappschuss in Abhängigkeit von `n`). Diskutieren Sie die in der Vorlesung genannten Nachteile von Schnappschuss-Algorithmen (hohes Nachrichtenaufkommen, „nicht verteilt genug“).

Visualisieren Sie einen konsistenten und einen inkonsistenten Schnitt anhand eines **Zeit-/Sequenzdiagramms** Ihrer konkreten Anwendung.

Abgabe

Eine erfolgreiche Abgabe dieses zweiten Teils der Portfolioprüfung besteht aus (1) dem **Sourcecode** zu den Aufgaben 1–3 (lauffähig in sim4da-S26), (2) einem **kurzen Bericht** (ca. 3–4 Seiten), der die gewählte Variante, die Behandlung der Kanal-/Nachrichten-Einfärbung sowie die Konsistenz-Experimente diskutiert und geeignete **UML-Diagramme** (Sequenz-/Zeitdiagramme) zur Visualisierung von Schnitten einsetzt, sowie (3) einem **ca. 10-minütigen Vortrag** in der Übung.

Abzugeben ist ein Repository mit Ihrer Implementierung einschließlich des Berichts als **Pull-Request** gegen das Github-Repository <https://github.com/syssoft-ds/2026S-DS.git>. Vermeiden Sie jegliche personenbezogenen Informationen (keine Matrikelnummer, kein Name), damit am Ende der Vorlesung alle Lösungen in diesem Repository zusammengefasst und veröffentlicht werden können. Die Zuordnung Github-Name zu Ihrem wirklichen Namen und Ihrer Matrikelnummer werden in der Übung fixiert.

Die Abgabefrist ist der 12.7.2026. Sollten bis zur Abgabefrist Fragen oder Probleme aufkommen, können Sie sich gerne über die üblichen Kommunikationswege an uns wenden.